

測る処理が、計算より重くなっていた — 1粒ずつ読むのをやめると 740倍速い

2026-09-19

何を作ったか

実験076で、同じ 15,591粒の MPM が、片方では 4.05秒、もう片方では 20秒かかった。違いは「毎フレーム、全部の粒を Python で読んでいたかどうか」だけだった。測るための処理が、測りたい計算より重くなっていた。

読み方を5通り用意して、実験070の動く板（40フレーム）で比べた。読み方ごとに毎回組み直し、計算と読む時間を別々に測った。

15,591粒で1フレームあたり、読まなければ 0.01ms。これまでの実験のやり方（1粒ずつ position() を読む）は 81.67ms で、計算そのもの（1フレーム約 86ms）とほぼ同じだけかかる。速さ v も1粒ずつ読むと 226.34ms で、計算の2.6倍。

全部を1回で受け取ると一気に軽くなる。pointFloatAttribValues は 0.57ms、文字列で受け取って numpy に渡すと 0.11ms。1粒ずつ読むのと比べて 740倍速い。読んだ値（高さの平均）は、どの読み方でも6桁まで同じだった。

粒が少なければ差は小さい。547粒では 1粒ずつでも1フレーム 2.94ms で、計算の数%にとどまる。粒を増やすほど、1粒ずつ読むやり方だけが重くなっていく。

15,591粒を1フレーム読む時間

「計算と比べて」は、40フレーム読む時間 ÷ 読まないときの計算の時間（3.457秒）。

実験077 — 15,591粒の位置を1フレーム読む時間

MPM の動く板（40フレーム）。計算そのものは 1フレーム約 86ms。

読み方	1フレーム	40フレーム	計算と比べて
1粒ずつ position() と v	226.34 ms	9.05 秒	2.6倍
1粒ずつ position()	81.67 ms	3.27 秒	0.95倍
pointFloatAttribValues	0.57 ms	0.023 秒	0.7%
numpy.frombuffer（文字列で受け取る）	0.11 ms	0.004 秒	0.1%

位置だけを読んだ3通りは、読んだ値（高さの平均 0.205530）が6桁まで同じ。

1粒ずつ読むと、計算と同じか、それ以上の時間がかかる。まとめて受け取れば 1% 未満。

粒の数と読み方（実験070の動く板・40フレーム）

「計算」は geometry() を呼んで計算させた時間、「読む」はそのあと値を取り出した時間。読み方ごとに組み直している。間隔 0.12 の「読まない」の読む時間 0.194秒は、最初の1回だけ numpy を読み込んだ時間（読み方の関数の中で import している）。

間隔	粒	読み方	計算の秒	読む秒	1フレームで読むms
0.12	547	読まない	2.226	0.194	4.86
0.12	547	1粒ずつ位置	1.571	0.118	2.94
0.12	547	1粒ずつ位置と v	1.575	0.326	8.15
0.12	547	まとめて	1.641	0.002	0.06
0.12	547	numpy	1.814	0.003	0.07
0.06	4580	読まない	2.813	0.000	0.01
0.06	4580	1粒ずつ位置	2.929	0.971	24.27
0.06	4580	1粒ずつ位置と v	3.316	2.726	68.15
0.06	4580	まとめて	2.835	0.007	0.19
0.06	4580	numpy	2.662	0.003	0.08
0.04	15591	読まない	3.457	0.000	0.01
0.04	15591	1粒ずつ位置	3.975	3.267	81.67
0.04	15591	1粒ずつ位置と v	4.872	9.054	226.34
0.04	15591	まとめて	3.575	0.023	0.57
0.04	15591	numpy	3.237	0.004	0.11

わかったこと

- ・1粒ずつ読むと、計算と同じだけの時間がかかる。15,591粒で1フレーム 81.67ms。計算そのもの（約 86ms）とほぼ同じ。速さ v も1粒ずつ読むと 226.34ms で、計算の2.6倍。
- ・まとめて受け取れば、ほぼ0になる。pointFloatAttribValues で 0.57ms、pointFloatAttribValuesAsString を numpy.frombuffer に渡すと 0.11ms。1粒ずつより 740倍速い。
- ・読んだ値は変わらない。位置だけを読んだ3通り（1粒ずつ・まとめて・numpy）で、最後の高さの平均は 0.338196 / 0.231387 / 0.205530（間隔 0.12 / 0.06 / 0.04）と、6桁まで同じだった。
- ・粒が少なければ気にしなくてよい。547粒では1粒ずつでも1フレーム 2.94ms。粒が 8.4倍（4,580）で 24.27ms、28.5倍（15,591）で 81.67ms と、粒の数にほぼ比例して重くなる。
- ・読む処理が重いと、計算の時間も少し伸びた。15,591粒で、読まないとき 3.457秒、1粒ずつ位置と v を読んだとき 4.872秒。なぜかは確かめていない（メモリの出し入れなどが考えられるが、推測）。
- ・実験076の「滑る塊は20秒かかった」の大半は、この読む時間だった。同じ粒の数の動く板は、最初と最後しか読まないの で 4.05秒だった。計算の重さを比べるときは、読むのは必要なフレームだけにする。
- ・計算の時間には、1回ごとに ±0.3～0.6秒ほどの揺れがある。間隔 0.12 で 1.57～2.23秒。0.1秒を争う比べ方をするなら、何度か測って中央値を取る。
- ・実用の結論。粒やポイントの値を Python で集計するときは、pointFloatAttribValues（できれば numpy.frombuffer）でまとめて受け取る。これまでの実験の測り方も、粒が多い場面ではこちらに切り替える。

次にやること

- ・hou_tools に「まとめて読む」関数を入れて、これまでの測り方を置き換える

- ・ ボリュームの値 (allVoxels / voxelSlice) でも同じ差が出るか
- ・ Liquid の暴れ始め (実験073) は、粒の細かさで動くか